



Claude Code

Upstage 초청 세미나

# AI에게 일 잘 시키는 기술

하네스 설계부터 Skill 활용까지

---

발표자  
김영동

일시  
2026년 5월 8일



PROFILE

김영동

Nkia · AI Engineer

인프라 장애 원인 분석(RCA)

에이전트 개발

[linkedin.com/in/youngdongkim](https://linkedin.com/in/youngdongkim)

UPSTAGE INVITED SEMINAR

# 목차

---

- 01 코딩은 어디로 가고 있는가
- 02 왜 Claude Code인가
- 03 AI 시대의 개발 방법론
- 04 하네스 & 컨텍스트 엔지니어링
- 05 이렇게 하면 망한다 — 한계와 실패 패턴
- 06 멀티 에이전트 활용
- 07 실전 워크플로우 & 도구 세팅
- 08 모두의 이야기
- 09 AI 시대, 우리는 어떻게 해야 하나
- 10 마무리 & Q&A

이 95장의 장표를  
한 장도 직접 만들지 않은 방법

한 것은 하나 — 하네스 설계

이것이 오늘의 목적지



01

# 코딩은 어디로 가고 있는가

---

## 추상화의 역사 — 1단계

기계어

```
01001000 01100101  
01101100 01101100  
01101111 00000000
```



```
printf("Hello");
```

## C 언어

*"기계를 직접 다루지 않으면  
개발자가 아니다"*

## 추상화의 역사 — 2단계

C

```
ptr = malloc(size);  
// ... use ptr ...  
free(ptr);
```



Java

```
Object obj = new Object();  
// ... use obj ...  
// GC handles cleanup
```

"메모리 직접 관리 안 하면 개발이 아니다"

추상화의 역사 — 3단계

```
print("Hello")
```

~~"장난감 언어, 실제 개발에 못 쓴다"~~

**Python → #1 언어**

추상화의 역사 — 그리고 지금

"이 함수를 리팩토링하고 테스트를 작성해줘"

~~"이건 진짜 개발이 아니다"~~

매번 빛나감

SECTION 1

## 숫자로 보는 변화

25%

YC 2025 겨울 배치 중  
코드베이스의 95%를 AI로 생성

~3개월

Meta 엔지니어가 코드를  
직접 타이핑하지 않은 기간

100%

AI가 짠 코드가 사람의 터치 없이  
PR에 그대로 올라오는 일상

## SECTION 1

# 마크다운이 코드를 대체하는 현실

이름은 다르지만 역할은 동일 — 마크다운으로 프로젝트 규칙을 AI에게 전달



**Claude Code**

`CLAUDE.md`



**Cursor**

`Cursor Rules`



**GitHub Copilot**

`copilot-instructions.md`



**Codex**

`AGENTS.md`



## SECTION 1

# 개발자의 새로운 역할

코드를 잘 짜는 능력



문서를 잘 쓰는 능력



컨텍스트를 설계하는 능력

비유로 보면

건축가

벽돌 쌓기 대신 설계도 작성

오케스트라 지휘자

직접 연주 대신 전체 조율

CLAUDE.md = 총보

## 여전히 남는 기초

"AI가 더 많이 해줄수록,  
기초 지식을 가진 사람의 경쟁력이  
역설적으로 올라간다"

변하는 것

### 직접 코드를 타이핑하는 비중

데모까지는 쉽지만, 실서비스로 만드는 건 전혀 다른 레벨

변하지 않는 것

### 시스템을 이해하는 능력

보안 · 동시접속 · DB 스키마 · 캐싱  
기본기 없이는 질문조차 불가

02

# 왜 Claude Code인가

---

## SECTION 2

# AI 코딩 도구의 진화 3단계

자동완성 → 대화형 → 에이전틱. 단계마다 컨텍스트 범위가 넓어짐

1단계

## 자동완성

IDE가 다음 줄 예측

한계 · 컨텍스트 제한



2단계

## 대화형 채팅

코드 토론 · 버그 설명

한계 · 다중 파일 수동



3단계 · 지금 여기 ★

## 에이전틱 코딩

목표 분해 · 독립 실행

전체 코드베이스 이해

## 개발 패러다임 전환

기존 개발

코드 작성

테스트 · 오류 · 수정

반복...

---

인간이 모든 단계를 직접 수행



에이전틱 개발

목표 정의

에이전트가 실행

변경 검토 · 승인

---

인간은 의도와 판단에 집중

## SECTION 2

# 에이전틱 코딩의 실제 성과

한 사례가 다 말한다 — 4~8개월짜리를 2주만에

# 2주

CTO 추정 **4~8개월** 프로젝트  
에이전틱 코딩으로 완료

Augment Code + Vertex AI

## 1~2일

신규 개발자 온보딩  
기존 **수주~수개월**에서 단축

### 복합 효과

불가능하던 작업 실현  
기술 부채 해결 · 즉각 피드백

## SECTION 2

## AI 코딩 도구 비교

| 도구                                                                                               | 형태       | 특징                      |
|--------------------------------------------------------------------------------------------------|----------|-------------------------|
|  GitHub Copilot | IDE 플러그인 | 코드 자동완성 중심, 가장 대중적      |
|  Cursor         | IDE      | AI 네이티브 에디터, 에이전틱 기능 내장 |
|  Windsurf       | IDE      | 에이전틱 IDE, Cascade 에이전트  |
|  Codex (OpenAI) | 터미널 CLI  | 에이전틱, 로컬 실행, 확장 레이어 구조  |
|  Claude Code    | 터미널 CLI  | 에이전틱, 로컬 실행, 확장 레이어 구조  |

## 에이전트 루프

목표만 주면 작업 완료까지 알아서 반복. 언제든지 중단하여 방향 수정 가능.





## SECTION 2

# 내장 도구 — 5가지 범주, 20개+

### 파일 작업

파일 읽기, 코드 편집, 새 파일 생성, 이름 변경 및 재구성

### 검색

패턴으로 파일 찾기, 정규식으로 콘텐츠 검색, 코드베이스 탐색

### 실행

셸 명령 실행, 서버 시작, 테스트 실행, git 사용

### 웹

웹 검색, 문서 가져오기, 오류 메시지 조회

### 코드 인텔리전스

타입 오류/경고 확인, 정의로 이동, 참조 찾기

## 확장 레이어 구조

CLAUDE.md만으로 시작. 필요에 따라 하나씩 추가.

기초

**CLAUDE.md**

모든 세션에서 로드되는 프로젝트 컨텍스트

자동화

**Skills**

재사용 가능한 워크플로우

**Hooks**

이벤트 기반 자동 실행

연결

**MCP**

외부 서비스 통합 — Notion, GitHub, Slack 등 100개+

확장

**Subagents**

격리된 병렬 작업

**Agent Teams**

독립 세션 협업

**Plugins**

패키징 · 배포

## SECTION 2

# Claude Code 생태계

도구 자체보다 **주변 생태계**가 더 빠르게 커지는 중.

### 공식 확장 포인트

#### 스킬 · 플러그인 · MCP · 후

Anthropic이 직접 설계한 확장 축 4종.  
공식 문서·마켓 제공.

### 오픈소스 커뮤니티

#### awesome-claude-code

GitHub 기반 플러그인·스킬·워크플로우  
모음. 레포·디스코드 활발.

### 한국 커뮤니티

#### 블로그 · 영상 · 세미나

한글 자료와 실무 사례가 빠르게 누적. 전  
사 도입에 필수 조건.

왜 생태계가 중요한가

도구의 기능 → 시간이 지나면 추격 가능.

커뮤니티가 만든 플러그인·스킬·노하우·교육 자료 → 복제 불가.

문제가 생겼을 때 검색하면 답이 나오는 환경

= 실무 도입의 핵심 조건

03

# AI 시대의 개발 방법론

---

### SECTION 3

## 왜 방법론이 필요한가

에이전틱 코딩 등장 → 병목의 이동.

이전의 병목

코드를 작성하는 것

지금의 병목

AI에게 **무엇을** 시킬지 정하는 것  
+ 정말 원하는 대로 했는지 **확인**하는 것

## 타임라인

2025.02

Karpathy, "Vibe Coding" 제시

AI 코딩의 대중화 · 문화적 전환점

2025 중반

AI 생성 코드의 품질·보안 문제 부각

구조화된 방법론의 필요성 대두

2025 하반기

SDD · 스펙 주도 프레임워크 등장

Vibe Coding의 한계에 대한 업계 대응

2026 초

TDD · Context Engineering 부상

성숙한 AI 개발 워크플로우의 정립

자유

엄격

**TDD**

테스트 주도

검증

**SDD**

스펙 주도

계획



## SECTION 3

# TDD (Test-Driven Development)

테스트가 먼저, 코드가 나중. AI 시대에는 인간이 테스트, AI가 구현.

1. **Red** · 인간이 실패 테스트 작성

2. **Green** · AI가 통과 코드 구현

3. **Refactor** · 인간 리뷰 → AI 리팩토링

버그 **40~80%** 감소 — 2025 DORA

### 왜 AI에게 특히 중요한가

AI는 "동작하는 것 같은" 코드를 자신 있게 만들.  
테스트 없으면 맞는지 틀린지 확인 불가.

### AI의 치팅에 주의

테스트 삭제 · 구현 먼저 작성 · assert 약화  
→ 테스트 수정 금지 규칙 필수

## TDD 예시

테스트를 먼저 쓰고, 통과하는 코드를 나중에 — AI 시대에도 원칙은 동일

"사용자가 운동 페이지에서  
과거 운동 클릭 시 상세 보기를  
볼 수 있어야 한다"

**RED** — 인간이 실패하는 테스트 작성

**GREEN** — AI가 최소 코드로 통과

**REFACTOR** — 인간 리뷰 후 AI가 정리

### AI 치팅 3대 패턴

- 테스트 삭제/skip으로 통과
- assert 약화 (값 → 타입 검증)
- 구현 먼저, 거기 맞춘 테스트

### CLAUDE.md 필수 규칙

#### 테스트 수정 금지

RED 실패 확인 전 GREEN 진행 금지

구조 변경과 기능 변경 커밋 분리

## SECTION 3

# SDD (Spec-Driven Development)

코드가 아닌 스펙이 진실의 원천. 무엇을(WHAT) 먼저 정의하고, 어떻게(HOW)는 AI에게.

### 1. SPECIFY — 인간이 "무엇을" 정의

요구사항 · 수용 기준 · 제약조건

### 2. PLAN — AI가 "어떻게" 제안

기술 스택 · 아키텍처 → 인간 승인

### 3. TASKS — 분해 후 병렬 실행

작업 ↔ 요구사항 매핑 유지

모호한 부분 → [NEEDS CLARIFICATION] 마커로 구현 중단

### 언제 쓰나

- 신규 기능 · 대규모 설계
- 여러 세션에 걸친 작업
- 팀 협업 (스펙 = 커뮤니케이션)

### 핵심

각 단계가 **파일로 저장 · 커밋 · 참조**  
→ 컨텍스트 소실 방어의 정석

## SECTION 3

# SDD 실전 — spec-kit 워크플로우

3개 커맨드로 스펙 → 계획 → 태스크 분해. 15분 안에 전체 구조 문서화.

### STEP 1 · SPECIFY

```
/speckit.specify "실시간 채팅 + 히스토리"
```

→ spec.md (WHAT/WHY 분리)

### STEP 2 · PLAN

```
/speckit.plan "WebSocket, PostgreSQL, Redis"
```

→ plan · data-model · contracts

### STEP 3 · TASKS

```
/speckit.tasks
```

→ 병렬 가능 작업 [P] 마킹

강제되는 원칙

- WHAT/WHY와 HOW 분리
- 모호 → [NEEDS CLARIFICATION]
- 테스트 가능한 수용 기준
- Constitution Check 게이트

핵심

모든 산출물이  
파일로 저장 · 커밋 · 참조

컨텍스트 소실 방어의 정석

출처: GitHub spec-kit

## 방법론 비교

|        | TDD              | SDD            |
|--------|------------------|----------------|
| 인간의 역할 | 테스트 작성자          | 스펙 작성자         |
| 검증 방식  | 자동 테스트           | 스펙 검증          |
| 최적 장면  | 기존 코드 수정 · 버그 수정 | 신규 기능 · 대규모 설계 |

가장 강력한 조합 — SDD로 정의 + TDD로 검증

결론

# Spec + TDD

## Spec

"무엇을" 정의  
의도 명확화

+

## TDD

"정말 그렇게 됐는지" 검증  
결과를 자동으로 확인

스펙 없는 TDD — 나무는 보되 숲은 놓침

TDD 없는 스펙 — 검증 수단 부재

이걸 매번 수동으로?

**스펙 템플릿**

계획 문서로 정착

**Skills**

TDD 워크플로우 패키징

**Hooks**

테스트 자동 실행

이 시스템이 곧 **하네스 엔지니어링**

04

# 하네스 & 컨텍스트 엔지니어링

---



## 엔지니어링의 진화

2023-

### 프롬프트 엔지니어링

모델에

무엇을 **말할** 것인가

---

단일 프롬프트 · 정적

2025-

### 컨텍스트 엔지니어링

모델에

무엇을 **보여줄** 것인가

---

입력 파이프라인 전체 · 동적

2026-

### 하네스 엔지니어링

모델 주변에

무엇을 **구축할** 것인가

---

실행 환경 전체 · 시스템 아키텍처

## SECTION 4

# 여전히 살아있는 프롬프트 엔지니어링

용어가 "컨텍스트 엔지니어링"으로 옮겨간 것뿐, 사라진 것이 아님. **기초에는 여전히 프롬프트.**

"프롬프트 엔지니어링은 죽지 않았습니다. 흡수되었을 뿐입니다. 좋은 컨텍스트 엔지니어는 여전히 좋은 프롬프트 엔지니어입니다."

— Phil Schmid, Hugging Face

**프롬프트**

한 번 말하는 법

C

**컨텍스트**

모이는 구조

C

**하네스**

실행 환경 전체

안쪽이 부실하면 바깥도 부실 — 모호한 CLAUDE.md 한 줄은 스킬·흑으로도 메울 수 없음

## 왜 프롬프트만으로는 부족한가

좋은 프롬프트로도 잔존하는 문제. 에이전트가 여러 단계를 거치는 순간 — 숫자가 불리해짐.

### 재현성 문제

같은 프롬프트에도 다른 결과.  
AI는 **비결정적** 시스템.

### 복리로 쌓이는 실패

한 단계 5% 실패율이  
20단계에선 치명적.

숫자로 보면

각 단계 성공률 **95%**

20단계 체인이라면?

**36%**

$$0.95^{20} = 0.358$$

"95% 동작" 에이전트 →  
실제 작업의 2/3에서 실패.

#### SECTION 4

## Agent = Model + Harness

AGENT  
**에이전트**

=

MODEL  
**텍스트 생성**

+

HARNESS  
**나머지 전부**

"모델이 아닌 것은 전부 하네스입니다." — LangChain, Vivek Trivedy

**비유 · 마구(馬具, horse tack)**

고삐 · 안장 · 재갈처럼, 강력하지만 예측 불가능한 말(= LLM)을 **올바른 방향으로 이끄는 장비 전체**가 하네스.

하네스가 결정하는 것 — **무엇을 보는지**(컨텍스트) · **무엇을 할 수 있는지**(도구·권한) · **언제 멈추는지**(제약) · **잘못됐을 때**(복구)

## 하네스의 6대 구성 요소

### 1 컨텍스트 엔지니어링

각 단계에서 모델이 보는 정보를 설계. 무엇을 유지·요약·버릴지 관리

### 2 도구 오케스트레이션

어떤 도구가 가용한지, 실행이 어떻게 처리되는지 결정

### 3 상태 및 메모리

수 분~수 시간 실행되는 에이전트의 내구성 있는 상태 관리

### 4 검증 루프

출력 생성 후 행동 실행 전 발동. 스키마·의미적·정책 유효성

### 5 오류 복구

실패 감지 → 재시도, 다른 접근법, 인간에게 풀백

### 6 인간 개입 제어

핵심 의사결정에서 일시정지. DB 삭제? 결제? → 승인 요구

— Kai Renner, harness-engineering.ai

## 에이전트 루프 — 하네스의 심장

거의 모든 코딩 에이전트가 반복하는 4단계. 각 단계가 하네스의 설계 지점.

### 1. GATHER CONTEXT

파일 읽기, 검색, 질문

무엇을 **안 보여줄지**를 설계하는 자리. 노이즈 유입 → 4가지 실패 모드 발동.

### 2. TAKE ACTION

코드 작성, 명령 실행, 툴 호출

가능한 한 **되돌릴 수 있는** 액션으로. 실패 비용 낮음 → 과감함 허용.

### 3. VERIFY WORK

테스트, 린트, 타입체크, 리뷰

자동 피드백 부재 → 루프가 **환각 강화**. 이 단계 막히면 전부 붕괴.

### 4. REPEAT

결과를 다음 루프 컨텍스트로

이전 결과가 다음 루프에 주입되는 지점. **Compaction·Pruning**이 필수.

하네스 엔지니어링 = 이 4단계의 각 지점을 신뢰성 있게 만드는 작업.

## SECTION 4

# Right Altitude — 지시문의 고도

CLAUDE.md나 프롬프트를 쓸 때 가장 흔한 실수는 두 극단.

### 너무 낮음 · OVER-SPECIFIED

모든 케이스를 if/else로 박기

→ 예외 상황에서 brittle

### 너무 높음 · UNDER-SPECIFIED

"잘 해줘", "깔끔하게"

→ 모델이 추측, 세션마다 결과 다름

### 올바른 고도 공식

원칙 1줄 + 이유(why) 1줄 + 예시 1~2개

이게 붙어야 새 상황에도 일반화 가능.

### 자가진단 2문항:

- ① 의도하지 않은 상황에서도 유효 → OK
- ② 아무 세션에 붙여도 행동 불변 → 너무 높음

## SECTION 4

# 하네스를 구성하는 4가지 도구

Claude Code가 제공하는 확장 축. 이 4가지를 조합해 하네스 구축.

### CLAUDE.md

프로젝트 규칙  
세션마다 자동 로드

### Skills

재사용 워크플로우  
/명령 으로 호출

### Hooks

이벤트 기반 자동화  
LLM 우회 — 강제 실행

### Plugins

팀 단위 배포 패키지  
npm install로 공유

**강제력 순서:** CLAUDE.md(권고) → Skills(호출 시) → Hooks(자동 차단) → Plugins(팀 배포)



## CLAUDE.md — 프로젝트의 "헌법"

모든 세션이 시작될 때 자동으로 로드되는 지속 컨텍스트. 하네스의 가장 기본적인 레이어.

### 먼저 짚고 갈 것 — "지시가 아니라 컨텍스트"

시스템 프롬프트가 아님 — 사용자 메시지로 주입.

강제가 아닌 권고. 모호하면 Claude가 임의 해석.

#### ✓ 검증 가능 · 구체적

- 슬라이드 규격: 720pt × 405pt
- 종결어미: 명사형·구 단위만
- 금지: ~합니다 / ~입니다 공손체
- 디자인 토큰: bg=#faf8f5

#### × 모호 · 검증 불가

깔끔한 코드 작성  
적절한 에러 핸들링  
좋은 패턴 사용

## SECTION 4

# CLAUDE.md 운영 원칙

공식 문서가 느낌표까지 붙어서 경고한 유일한 원칙 — **짧을수록 효과적.**

### 한 줄 삭제 테스트

"한 줄씩 지워보세요. 지워도 Claude가 똑같이 동작하면 — 그 줄은 잡음."

— Claude Code Best Practices

### 크기 가이드라인

~60줄 · HumanLayer (엄격파)

A4 약 1.5장 · Boris Cherny 팀

200줄 이하 · 공식 문서

500줄 이하 · Marconato

숫자보다 **신호 대 잡음비**. 같은 신호 → 짧을수록 우세.

### 비대해지면 — 모듈화

`@path/to/file` — 다른 파일을 import (최대 5호)

`.claude/rules/` — 경로별 규칙, 읽을 때만 트리거

`/compact` 후에도 CLAUDE.md는 재로드

#### SECTION 4

## CLAUDE.md는 살아있는 문서 — 현장 사례

한 번 쓰고 끝나는 정적 문서가 아닌, **피드백으로 자라는 유기체**처럼 운영.

### Boris Cherny

Claude Code 창시자 · Anthropic

주 수 회 업데이트.

**Claude가 실수할 때마다 한 줄 추가.**

PR에서 `@claude` 태그 → GitHub Action이 자동 커밋.

### 정구봉 (Team Attention)

규모의 실천

**CLAUDE.md 159개** + 스킬 100개+ 동시 운영.

"파일 하나는 짧게, 대신 파일을 많이."

**"버그는 한 번 고치는 게 아니라, CLAUDE.md에 한 줄을 추가하면서 고치는 것이다."**

## SECTION 4

# Skills & Hooks — 그게 뭐가

하네스를 구성하는 두 실행 단위. **Skill** = 부르면 실행되는 워크플로우 · **Hook** = 이벤트에 자동 실행되는 스크립트.

### SKILL

#### 재사용 워크플로우 패키지

**언제** — 반복되는 작업을 한 번에 호출하고 싶을 때

**형식** — `frontmatter(name · description) + markdown 본문`

**호출** — `/skillname` 또는 자연어 트리거

**선택** — `description` 기반 자동 매칭, 토큰 소비 없음 (호출 시 로드)

예: `/commit` · `/review-pr`

### HOOK

#### 이벤트 기반 자동 스크립트

**언제** — 항상 적용해야 하는 규칙·검증·차단

**형식** — 이벤트 타입(`PreToolUse...`) + 실행 커맨드

**실행** — LLM이 아닌 쉘 스크립트 — 반드시 실행

예: 파일 편집 후 **ESLint** · 커밋 전 **테스트** · `.env` 편집 차단

**구분법** — **Skill** = 내가 부를 때 실행 · **Hook** = 자동 실행.

## SECTION 4

# Skill은 어떻게 동작하는가

## 실제 파일 구조

```
.claude/skills/commit.md

---
name: commit
description: Git 커밋 시 팀 컨벤션
  메시지 자동 생성
---

# 규칙
1. 브랜치명에서 이슈 번호 추출
2. 변경사항 분석 → Type 결정
3. #{ISSUE} {Type} : {설명}
```

frontmatter의 description이 핵심 — Claude가 이걸 읽고 매칭 결정

## 호출 → 실행 흐름

1 사용자: `/commit` 또는 "커밋해줘"

2 Claude가 등록된 skill의 **description** 스캔

3 매칭된 skill 본문을 컨텍스트에 로드

→ 이 시점에서만 토큰 소비 (평균 0)

4 본문의 지시대로 작업 수행

Skill = 필요할 때만 꺼내 쓰는 **CLAUDE.md**

100개 등록해도 호출 전엔 토큰 0

## SECTION 4

# Plugins — 하네스를 포장해서 팀 자산으로

4개 레이어 위에 얹히는 배포 레이어

새 기능이 아니라 포장지

이미 존재하는 **Skill · Subagent · Hook · MCP · LSP**를 단일 단위로 번들링해 팀·커뮤니티에 배포하는 메커니즘

```
my-plugin/  
├── .claude-plugin/plugin.json  
├── commands/ agents/  
├── skills/ hooks/  
└── .mcp.json
```

standalone → plugin 전환은 파일 이동 + 매니페스트 1개 수준

우리 팀 플러그인 설치 (실제)

```
claude - /plugin  
  
> /plugin marketplace add nkia-ai-team/claude-code-skills  
✓ Added marketplace: nkia-ai-marketplace  
→ 14 skills available  
  
> /plugin install nkia-ai-tools@nkia-ai-marketplace  
✓ Installing nkia-ai-tools ... done  
→ Added skills: /commit /code-review /kickoff ...  
  
> /plugin list  
nkia-ai-tools ● installed
```

개인이 쌓아올린 하네스를 **조직 자산**으로 전환하는 마지막 한 조각

## SECTION 4

# 우리 팀의 스킬 레포

AI 1팀에서 운영 중인 14개 스킬. 반복되는 워크플로우를 `/명령` 한 줄로.

### 코드 품질

`/code-review` — PR/MR 자동 리뷰

(OWASP · SOLID · 성능 · 테스트)

`/commit` — 이슈 번호 포함 · 컨벤션 커밋 자동 생성

### 프로젝트 관리

`/linear-issue-creator` — 자연어 → 이슈 자동 생성

(DoD · AC · 마감일 · 라벨)

`/linear-issue-validator` — DoD/AC 자동 검증

### 온보딩 · 전환

`/kickoff` — 프로젝트 초기 세팅

`/figma-to-react` — Figma → React 변환

### 워크플로우

`/submit` — 작업 제출 + PR 생성

`/wrap-up` — 일일 작업 정리 · 보고

한 번 만든 스킬 → 팀 전원이 `/commit` 한 마디로 호출. **반복 작업의 종결.**

SECTION 4

도구를 알았으니

# 조합하는 패턴

Plan-Critic-Build

Ralph Loop

암묵지 추출



## SECTION 4

# Plan-Critic-Build — 계획과 실행의 분리

에이전트가 가장 많이 실패하는 지점: **계획 없이 바로 코드 작성**. 해결책은 단순 — 계획 단계를 쓰기 권한 없는 모드로 분리.

## 1. PLAN

### 읽기만 · 쓰기 금지

파일 읽기·검색·질문만. 결과: 편집 가능한 계획 문서.

## 2. CRITIC

### 다른 눈의 시각

리뷰 전용 에이전트. 작성자의 blind spot을 드러내기.

## 3. BUILD

### 승인 후 실행

승인된 계획 링크만 컨텍스트로. 쓰기 권한 개방.

**핵심: Planner와 Critic을 같은 컨텍스트에 두지 않기.** 자기 계획 → 자기 비평 시 blind spot 잔존.

Claude Code의 Plan Mode( `Shift+Tab` `Shift+Tab` ) — 1단계를 구조적으로 강제.

*"계획과 실행을 분리하는 것이 제가 AI 코딩에서 하는 가장 중요한 행위이다."* — Armin Ronacher

## SECTION 4

# Ralph Loop — 가장 단순한 하네스



PROMPT.md 하나를 무한 루프로 반복 실행. The Simpsons의 Ralph Wiggum에서 이름을 딴 — "바보처럼 단순한" 루프.



단순성 = 예측 가능성

새 프로젝트 전용

검증 없으면 환각 누적

## 암묵지를 파일로 뽑아내라

"뭘 보여줘야 하지?" — 답은 이미 머릿속에.

리뷰할 때 말하던 것, 버그 겪고 반사적으로 체크하던 것 → 파일로 추출.

- ① **결정의 "이유"까지 기록** — 결과만 → 금방 낡음. 이유까지 → 일반화 가능
- ② **Post-Mortem** → 규칙 — 계획과 구현의 분기점을 한 줄로 승화
- ③ **Self-Improving** — 피드백마다 규칙 한 줄씩 성장
- ④ **취향** → 전용 에이전트 — 글로 쓰기 어려운 판단을 에이전트로 encode

머릿속 판단이 파일로 옮겨지는 순간 **하네스가 된다** — 한 줄씩 박제.

경험 → 규칙 → 재사용. 모호함도 점수화(0~10) → 구조화 가능.

## SECTION 4

# Harness Builder — 새로운 역할

## Builder

기능을 구현하고  
코드를 작성

## Reviewer

코드의 품질, 정확성,  
보안을 검토

## Harness Builder

에이전트가 일하는  
환경 자체를 설계

OpenAI Codex 팀 — 5개월간 수동 코드 0줄로 100만 줄 프로젝트

"우리 팀의 주된 업무는 에이전트가 유용한 일을 할 수 있도록 **환경을 만드는 것이 되었다.**" — OpenAI Codex 팀

1 리포지토리 지식 체계 설계

2 결정론적 제약 구현 (린터, 테스트)

3 에이전트 가독성 최적화

4 취향과 원칙의 인코딩

05

# 이렇게 하면 망한다

한계와 실패 패턴



## SECTION 5

## 신뢰성의 수학 — 복리로 쌓이는 실패

각 단계 성공률 **95%** 가정 → 다단계 작업의 전체 성공률?

| 단계 수         | 전체 성공률     |
|--------------|------------|
| 1 단계         | 95%        |
| 5 단계         | 77%        |
| 10 단계        | 60%        |
| <b>20 단계</b> | <b>36%</b> |
| <b>50 단계</b> | <b>8%</b>  |

계획 → 탐색 → 작성 → 테스트 → 수정 → 커밋 — 20단계만 가도  
**2/3이 실패**

에이전트를 만드는 건 쉬운 부분 — 프로덕션 신뢰성이 진짜 엔지니어링

검증 루프 · 재시도 · 체크포인트 — 복합 실패율을 낮추는 건 **하네스**뿐

## SECTION 5

# 컨텍스트의 4가지 실패 모드

"많이 넣어도 OK" — 사실 아님. Drew Breunig가 정리한 4가지 실패 모드.

### POISONING · 중독

환각 1회 → 이후 오류 누적

초반 오답 1개로 평균 39% 성능 저하

### DISTRACTION · 산만

히스토리 과의존, 실패한 액션 재시도

### CONFUSION · 혼동

무관한 정보 과잉 → 톨 30개+에서 급증

### CLASH · 충돌

모순된 정보 → 추론 붕괴

*"컨텍스트를 잡동사니 서랍처럼 다루면, 그 잡동사니가 답변에 영향을 줍니다."* — Drew Breunig

핵심은 "뭘 넣을까"보다 "뭘 뺄까".

## SECTION 5

# Context Rot — 길수록 조용한 부패

Chroma Research가 Claude · GPT · Gemini · Qwen 등 **18개 최신 모델**을 대상으로 측정한 결과.

## 50k 토큰

20만 토큰 모델이 이미 열화되는 지점

한국어 약 4만 자. 윈도우가 짝 차기 훨씬 전부터 성능 하락.

## 방해문장 1개

하나만 섞여도 성능 저하

4개면 급감. "많이 넣으면 좋다" 직관 붕괴.

"관련 정보가 컨텍스트에 있느냐가 전부는 아닙니다. 더 중요한 것은 그 정보가 **어떻게** 제시되느냐입니다."

— Chroma Research

컨텍스트 = 길이가 아닌 신호 대 잡음비. 쓸 수 있는 만큼 다 채울 필요 없음.



## 실패 패턴 5가지

도구를 잘 쓰는 것만큼 중요한 건, **잘못 쓰는 패턴**을 아는 것.

### 01 컨텍스트 과부하

"많이 주면 잘하겠지" — 섹션 4의 Rot · Distraction · Confusion

### 02 불명확한 요구사항

"이거 좀 고쳐줘" → LLM은 학습 데이터 기반 **추측 구현**. 맞아도 재현성 없음

### 03 검증 없는 자동수락

"AI가 만들었으니 맞겠지" — 가장 위험한 생각. 자기 작업 평가 시 **병리적 낙관주의자**

### 04 긴 세션 드리프트

시점 편향 · 오류 중첩 · Mode Collapse — 초반 규칙을 잊고 방향 상실

### 05 권한 과다

"바이패스가 편하니까" — 프롬프트 인젝션 · 토큰 유출 · 프로덕션 사고의 단일 경로

## 그래도 여전히 중요한 기초

"AI가 다 해주니까 코드 안 배워도 되겠다"

— 가장 위험한 생각

AI의 자신감 있게 틀린 코드 제시 — 기초 없으면 틀림 감지 불가

추상화 수준만 올라갔을 뿐, 문제 해결 · 시스템 사고 · 디버깅은 여전히 필수

진입장벽은 낮아졌으나, 고품질 역량은 오히려 더 구조화

06

# 멀티 에이전트 활용

---

## SECTION 6

# 단일 에이전트의 세 가지 벽

하나의 에이전트가 오래 일할수록 필연적으로 맞닥뜨리는 벽 — **또개야 할 이유.**

## 01 컨텍스트의 벽

파일 · 테스트 · 로그 왕복 → **쓰레기로 가득한 컨텍스트**

*50k 토큰부터 Context Rot · Distraction · Confusion 발동*

## 02 역할의 벽

"쓰고, 구현하고, 리뷰하고, 리팩터하라"를 한 에이전트에게 → **전부 어설프게.**

*자기 코드 자기 리뷰 = blind spot 그대로*

## 03 신뢰성의 벽

단계별 95% × 20단계 연쇄 = **전체 성공률 36%.**

*긴 체인 = 수학적 실패*

하나의 에이전트 = **하나의 일.**

## SECTION 6

# 핵심 패턴 5가지

무엇을, 어떻게 **쫓개느냐**에 대한 서로 다른 답.

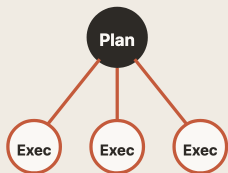
### 01 Sub-Agent



#### 단방향 심부름

탐색 · 검증 · 로그 파싱

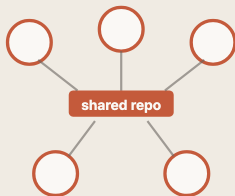
### 02 Orchestrator



#### 1:N 위임

계획자 + 실행자들

### 03 Parallel



#### 평면 조직

Git worktree 격리

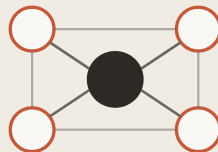
### 04 GAN-Style



#### 적대적 루프

루브릭으로 수렴

### 05 Agent Teams



#### 양방향 대화

그래프 구조

## Subagent vs Agent Team

둘 다 작업 위임 방식 — **규모와 독립성**에 따라 선택.

### Subagent

#### 구조

메인이 생성한 격리된 워커

#### 컨텍스트

자체 윈도우, 요약만 반환

#### 통신

주 에이전트에게만 보고

---

많은 파일 읽기, 병렬 조사

### Agent Team

#### 구조

여러 독립 세션이 협업

#### 컨텍스트

완전히 독립, 각자 별도 윈도우

#### 통신

팀원끼리 직접 메시지

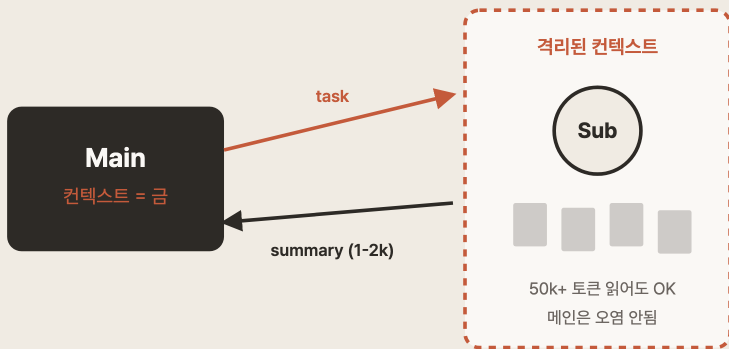
---

보안/성능/테스트 동시 검토

## SECTION 6

# Sub-Agent — 가장 자주 쓰는 패턴

일상 작업의 80% — Sub-Agent로 해결.



### 컨텍스트 격리

파일 50개 뒤져도 메인 오염 없음

### 도구 제한

읽기 전용만 허용 → 사고 원천 차단

### 정보 격리 = 치팅 차단

Tester는 구현 못 봄 · Impl은 스펙 못 봄

메인 컨텍스트 = 금 · 서브에이전트의 산출 = 요약

## SECTION 6

# Agent Teams — 양방향 대화가 가능한 팀

Sub-Agent가 "시켜놓고 받는" 구조라면, Teams는 **같이 토론하고 합의하는** 구조.

### 동적 팀 구성 예 — 경쟁사 분석



### 자동 수정 루프

Worker 구현 → QA 검증 → Support 수정 → QA 재검증

### 공유 컨텍스트 बैं크

프로젝트 스펙·제약은 중앙에 두고 참조

3명까지는 이득 · 10명 넘으면 대기 시간 > 작업 시간 — **"팀이면 다 좋다"**는 환상 경계.



## 설계 4대 원칙

패턴이 무엇이든 공통으로 지켜야 하는 것 — **쫄갠의 네 축**.

01

### 컨텍스트 격리

각 에이전트 = 자기 일에 필요한 것만 · 메인 = 결정과 요약만.

02

### 역할 경계 명시

"무엇을 하고 무엇을 안 하는지" 명시 · 모호함 → 중복과 공백.

03

### 사전 모호성 제거

실행 후 발견 → 이미 늦음. Ralphon 우승팀 — 개발 전 **133회 Q&A**.

04

### 검증 주체 분리

생성자 ≠ 검증자. Self-review → blind spot 잔존.

멀티 에이전트 = **"더 많은 AI"가 아니라 "더 좁은 역할의 AI 여럿"**

컨텍스트를 쫄갠다 · 역할을 쫄갠다 · 검증을 쫄갠다 · 시간을 쫄갠다

07

# 실전 워크플로우 & 도구 세팅

---

## 두 가지 막다른 길

하네스 · 에이전트 · 컨텍스트 방어 · 암묵지 문서화 — 배울 것이 많음.

### 직접 다 세팅

- 한 번 세팅해도 업데이트 안 됨
- 팀에 공유되지 않음
- 시간이 지나면 맥락 휘발
- → **번아웃**

### 원리 없이 스킬만

- 이상 동작 시 원인 판단 불가
- 같은 실수를 반복
- 디버깅할 수 없음
- → **블랙박스 의존**

**원리는 이해 · 구현은 위임** — 잘 만들어진 플러그인 활용.

# OMC — 원리를 명령어로

Oh My Claude Code — Yeachan Heo(한국) 제작, GitHub Trending 1위. 핵심 — 원리를 사람이 외우지 않고 시스템이 강제

## 원리

## OMC 구현

역할 분리

29+ 에이전트 — architect · critic · tester ...

모델 라우팅

`/model` 수동 전환 · 자동 선택 · 토큰 **30~50% ↓**

Plan-Critic-Build

`/ralplan` — 계획 → 비평 → 합의 후 실행

병렬 실행

`/ultrapilot` — 최대 5 워커 동시

검증 루프

`/ultraqa` — test → fix → 재실행 자동 순환

Worktree 병렬

`/project-session-manager` + HUD + Slack

핵심 — 사용자는 "어떤 작업인지"만 입력. 모드 · 에이전트 · 모델은 OMC 자동 선택.

## 한 걸음 더 — 단일 모델 탈피

섹션 6이 여러 Claude였다면, 이건 여러 종류의 AI. 모델마다 상이한 훈련 분포·blind spot.

### /ask [provider]

Claude 답이 불확실할 때 다른 모델에 의견 요청

```
# 구조 리뷰를 Codex에게
/ask codex "이 계획에서 짤 구조
리뷰해줘. 빠진 부분 있는지"

# 디자인 자문을 Gemini에게
/ask gemini "이 카드 레이아웃
가독성 어떤지 피드백 줘"
```

### /ccg — Claude · Codex · Gemini

세 모델 동시 송출 → 합의·불일치·불확실 분리 표시

#### 특히 빛나는 순간

- 아키텍처 결정 (되돌리기 어려움)
- 디버깅이 막혔을 때
- 머지 전 코드 리뷰 최종 확인
- 계획 피드백 (다른 시각의 보완)

## SECTION 7

# Upstage = 모델 + 입력 계약화

단일 API가 아닌 — LLM 모델과 도메인 특화 Skills 두 축으로 한국어 컨텍스트 공략

### AXIS 1 · MODEL

## Solar (LLM)

#### solar-pro3

한국어 long-form 생성 · 추론

#### solar-mini

경량 · 저비용 작업

### AXIS 2 · SKILLS (입력 계약화)

## Document Intelligence APIs

#### document-parse

PDF → 구조 보존 마크다운

#### information-extract

스키마 기반 데이터 추출

#### ocr

단순 텍스트 추출

#### document-classification

자동 분류 · 라우팅

한국어 컨텍스트 = 모델(생성) + Skills(입력 계약화) — 양쪽 다 필요

## SECTION 7

# 같은 PDF, 두 가지 마크다운

발표 PDF 89장 · 같은 명령 · 다른 도구 — Upstage 스킴 검증 실험

그냥 시켰을 때

"마크다운으로 만들어줘"

Claude의 자동 선택

**macOS Vision OCR**

이미지 PDF · 텍스트 레이어 0

UPSTAGE 스킴

"마크다운으로 만들어줘"

스킴 명시

**document-parse API**

단일 엔드포인트 · layout 인식 포함

# 차이는 한 슬라이드에서 갈림

Slide 2 · 발표 목차 페이지 — 실제 추출 결과 비교

## × macOS Vision OCR — parse-default.md

```
01
코딩은 어디로 가고 있는가
02
왜 Claude Code인가
03
AI 시대의 개발 방법론
04
하네스 & 컨텍스트 엔지니어링
05
이렇게 하면 망한다 - 한계와 실패 패턴
...
```

번호·제목 분리 · 표 구조 소실

## ✓ UPSTAGE document-parse — parse-upstage.md

```
| 01 | 코딩은 어디로 가고 있는가 |
| --- | --- |
| 02 | 왜 Claude Code인가 |
| 03 | AI 시대의 개발 방법론 |
| 04 | 하네스 & 컨텍스트 엔지니어링 |
| 05 | 이렇게 하면 망한다 |
| ... | ... |
```

번호·제목 매핑 보존 · 마크다운 표 그대로

OCR 능력 = 비숫 · **차이 = layout 이해력** · 발표 PDF 89장 전체에 같은 패턴



## SECTION 7

# OCR 다음의 한 단계

글자 추출 → layout 분석 → 구조 재구성

### 일반 OCR 파이프라인

PDF



이미지 변환



OCR · 글자 + 좌표



한 줄씩 나열

글자만

### UPSTAGE 파이프라인

PDF



OCR · 글자 + 좌표



★ Layout 분석



표·차트·헤딩 분류

글자 + 구조

글자만 = LLM이 추측 · 구조까지 = LLM이 이해

스킬 = 입력 계약화 = 컨텍스트 엔지니어링

# Solar 위임 — Claude는 두뇌, Solar는 글쟁이

**upstage-solar-delegation** 스킴 — 긴 한국어 출력 자동 분배. 임계값  $\geq 200$  토큰

## 위임 흐름

1. **Claude** — 계획 · 도구 호출 · 자료 수집
2. 출력 분량 추정 → 임계값( $\geq 200$  토큰) 초과?
3. 자동 spawn → **solar-pro3** 세션
4. **Solar** — 한국어 long-form 생성
5. 사용자에게 Solar 출력 그대로 반환

사용 예

"한국어 가이드 5천자 작성" → Claude 계획·자료 수집 → Solar 본문 생성

## OUTPUT TOKEN 비용

solar-pro3 vs Claude Opus 4.7

# 1/42

\$ / 1M output tokens

Solar Pro 3 — **\$0.60**

Opus 4.7 — **\$25.00**

5천자 한국어 보고서: Opus \$0.10 → Solar \$0.0024

multi-model의 진화형 — **사람이 매번 모델 고를 필요 없음**. 시스템이 강점에 따라 자동 분배

## SECTION 7

# 실전 세팅 — 다중 세션 병렬

cmux(AI 전용 터미널) + Git Worktree — 에이전트 3~4개를 동시에 돌리는 물리적 환경

### 창 1 · 메인 작업

현재 이슈 구현·수정

Opus 모델 · 핵심 컨텍스트 보호

### 창 2 · 조사·탐색

코드베이스 탐색, 문서 검색

Explore 서브에이전트 역할

### 창 3 · 감시

테스트·빌드 로그 실시간 확인

에이전트 완료 알림 수신

### 창 4 · 이슈·PR

Linear 이슈 확인, PR 생성

git 상태·브랜치 관리

### Git Worktree — 창마다 독립 작업 디렉토리

```
claude --worktree feature-auth
```

 한 줄로 워크트리 + 브랜치 + 세션 동시 생성. 파일 충돌 0, 컨텍스트 오염 0.

핵심 — 메인 컨텍스트를 오염시키지 않는 것. 탐색은 서브 창에서, 결과만 메인에 전달.

## SECTION 7

# 이슈 기반 워크플로우 — Linear에서 PR까지

이슈 1개 = 에이전트 세션 1개. 이슈 트래커가 곧 에이전트의 작업 지시서.

Linear 이슈

→

Worktree 생성

→

Claude 세션

→

PR 생성

→

검증·완료

### AI 1팀 스킬 체인

`issue-creator` 구조화된 이슈 생성

`issue-evidence` AC별 증거 첨부

`issue-validator` DoD·AC 자동 검증

### 왜 이슈 기반인가

- 채팅 = 휘발. 이슈 = 지속적 컨텍스트
- AC가 명확하면 에이전트가 완료 판단 가능
- 이슈 단위 = 컨텍스트 드리프트 차단

Linear — 개발팀 이슈 트래커. MCP 연결로 터미널에서 **이슈 → 코드 → PR → 검증** 전 과정 완결.

08

# 모두의 이야기

---

기억하시나요

이 발표를  
코드 한 줄 없이 만든 방법

한 것은 하나 — 하네스 설계

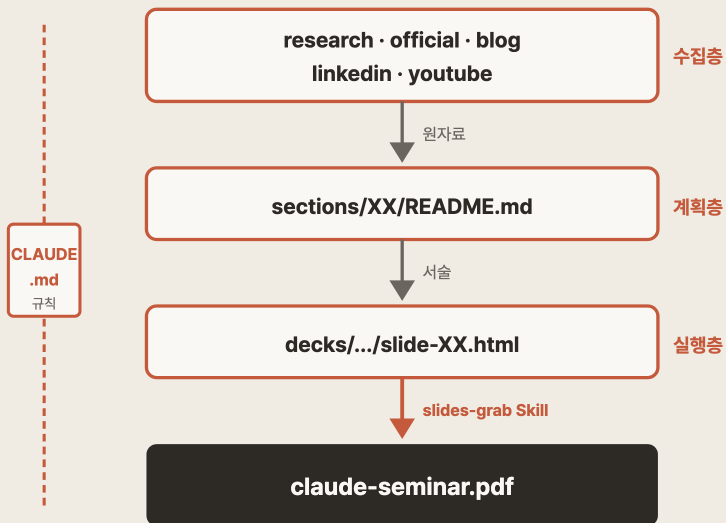
지금부터 그 증거

## SECTION 8

# 이 발표가 만들어진 과정

김영동 + Claude — 마크다운과 규칙 문서만으로 95장 제작

파이프라인 — 3층 + CLAUDE.md 하네스



앞 섹션의 원리 그대로

Spec-first · 하네스 · Skill 재사용

특별한 기술 없음

마크다운 + HTML + 규칙 문서

앞에서 다룬 원리 — 그대로 적용된 결과물

## 재료 1 — 폴더 구조로 박힌 분리

수집층 · 계획층 · 실행층이 **폴더 경계로 분리**. 규칙이 아닌 파일 구조로 강제.

```

claude-seminar/
├── research/ # 하네스·컨텍스트 리서치
├── official/ # Claude Code 공식 문서
├── blog/ # Anthropic + 한국 커뮤니티
├── linkedin/ # 실무자 인사이트
├── youtube/ # 영상 요약
├──
├── sections/ # 9개 섹션 마크다운 (계획)
├── decks/claude-seminar/ # 슬라이드 (실행)
│   └── slide-01.html ~ slide-91.html
├──
└── CLAUDE.md # 프로젝트 규칙
  
```

### 수집층 = 근거의 정박지

"이 주장 어디서?" 질문에 → 파일이 답

### 계획층 ≠ 실행층

sections/에 본문 먼저, decks/는 디자인만 — 섹션 4의 Plan-Critic-Build 강제

근거 없이 작성 → **환각이 장표로 박제**



## 자료 2 — CLAUDE.md가 만든 일관성

디자인 감각·카피 원칙이 문서에 축적 · 새 슬라이드가 자동 추종

### 초기 CLAUDE.md (v1)

- 슬라이드 규격: 720pt × 405pt
- 폰트: Pretendard (CDN)
- 디자인 토큰 6개

### 진화 과정

초반: ~합니다 공손체 → 스캔 불가

중반: ~다 서술형 → 여전히 읽어야

결론: 명사형 통일 → CLAUDE.md에 고정

### 현재 CLAUDE.md (피드백 5회 반영)

- + 종결어미: 명사형·구 단위만
- + 금지: 공손체·서술형·명령형
- + 카드 레이아웃 규칙 추가
- + 강조 체계 3단계 정의
- + 피드백 관리 워크플로우

95장에 걸쳐 카피 톤이 일관된 이유 —

사람의 89번 검토가 아닌, **한 번 쓴 규칙의 결과**

## SECTION 8

# 자료 3 — slides-grab, Skill 경계로 박힌 분리

**slides-grab** — 오픈소스 Claude Code용 프레젠테이션 Skill. 아웃라인 → HTML 슬라이드 → PDF를 **3단으로 분리**.

### STAGE 1 · PLAN

#### slides-grab-plan

`slide-outline.md` 생성

사용자 승인까지 반복

### STAGE 2 · DESIGN

#### slides-grab-design

승인된 아웃라인 →

`slide-XX.html` 생성·수정

### STAGE 3 · EXPORT

#### slides-grab-export

HTML → PDF 변환

산출물 검증

**왜 3단인가** — 단계 사이에 사용자 승인을 **물리적으로 강제**. Plan에서 방향이 틀어지면 Design-Export 비용 전부 낭비.

Plan → Design → Export 사이에 **사람의 승인이 물리적으로 끼어듦** — 건너뛸 수 없는 관문.

## 이 발표가 증거

이 장표 95장 전부 — Claude Code로 제작. HTML 한 줄 직접 작성하지 않음.

### 사용한 원칙 — 이 발표에서 다룬 것 그대로

**Spec-first** — sections/README.md에 내용 먼저

**CLAUDE.md** — 디자인 토큰 · 호환 규칙 · 카피 원칙 고정

검증 분리 — 피드백 → pending.md → 검토 후 반영

자동화 — slides-grab으로 HTML → PDF 일괄 변환

### 실제 워크플로우

- 1 섹션별 내용을 자연어로 정리
- 2 CLAUDE.md에 디자인 규칙 명세
- 3 Claude Code가 slide-XX.html 생성
- 4 피드백 8회 반복 → 최종 PDF

규칙을 하네스로 고정 → 실행은 에이전트에게 → 결과를 검수. **이 발표의 제작 과정 자체가 증거.**

## SECTION 8

# AI를 잘 쓰는 법 — 결국 하나

코드 · 문서 · 기획 · 디자인 — 도메인은 달라도 원칙은 같음.

원하는 것을  
글로 먼저



규칙을  
문서로 고정



결과를  
사람이 검증

### 이 발표

섹션 README → CLAUDE.md → 피드백 검수

### 코딩

Spec → 하네스 규칙 → 테스트 검증

### 업무 전반

요구사항 정리 → 기준 문서 → 결과 확인

새로 배울 것 없음 — 이미 알고 있는 원칙을 AI 시대에 더 의식적으로

09

# AI 시대, 우리는 어떻게 해야 하나

---

## 최전선의 피로 — 똑같이 느끼는 그들

"지식이 쌓이는 속도보다 감가상각되는 속도가 더 빠르다. 나도 지쳤다."

— Simon Willison, 2025

### FOMO-AI · 2025 학계 척도

- 뒤처질까 두려움 (backwardness)
- 좋은 도구를 못 쓸까 두려움 (access)
- 남만 이득 볼까 두려움 (dividend)

### 논문의 결론

FOMO를 가장 크게 줄이는 것 — 더 많은 정보가 아니라 **직접 써보기**

맨 앞의 사람들조차 같은 피로 — 다른 점은 "더 빨리" 대신 "무엇을 남길지 고르기"에 시간 투자

## SECTION 9

# 증폭되는 경험 — 세 개의 증언

50년 넘게 코드를 쓴 사람, 최전선에 있는 사람, 업계의 대부 — 세 사람의 거의 같은 메시지.

### KENT BECK

TDD 창시자 · 52년

"프로그래밍은 여전히 프로그래밍이다. 어떤 면에서는 훨씬 더 나은 경험이다. 시간당 더 중요한 결정을 내리게 된다."

### SIMON WILLISON

Vibe Engineering

"AI는 기존 전문성을 증폭시킨다. 더 많은 기술과 경험을 가질수록 LLM으로부터 더 빠르고 더 좋은 결과를 얻는다."

### MARTIN FOWLER

Thoughtworks

"AI는 주니어의 산출물은 복제할 수 있지만, **시니어의 경험과 판단은 복제할 수 없다.**"

## AI 시대의 경험 — 축소가 아니라 증폭

Spec-first · TDD · Plan-Critic · 검증 분리 — 시니어가 원래 하던 것. AI는 그걸 할 줄 아는 사람에게 주는 레버리지

## 내일부터

### 글로 먼저

CLAUDE.md 1페이지 작성, 또는 반복 업무 1개를 글로 명세

### 직접 써보기

터미널 열고 Claude Code 3~4창 — 토큰을 일부러라도 소비

### 도구

Oh My Claude Code 설치 — 원리의 자동화

안 쓰는 것보다 써보는 게 발전. "토큰을 많이 쓰는 사람이 가장 경쟁력 있는 개발자" — 박종천





Claude Code

# 감사합니다

AI 1팀 김영동

---

APPENDIX

# 부록

추천 채널 & 참고 자료



## 추천 채널 & 계정

AI 개발 트렌드를 따라가기 위한 정보 소스

### YOUTUBE

#### 빌더 조쉬

하네스 엔지니어링  
AI 네이티브 조직 설계

### LINKEDIN

#### 정구봉 (gb-jeong)

Team Attention  
에이전트 · 컨택스트

### LINKEDIN

#### 이재영 (jyoung105)

AI 엔지니어링  
최신 트렌드 큐레이션

### COMMUNITY

#### GeekNews

한국 개발자  
뉴스 큐레이션

### COMMUNITY

#### Claude Code

Discord  
실전 사용 사례 공유

### COMMUNITY

#### OpenAI Developers

Discord  
AI 생태계 전반

### 전체 참고 자료

공식 블로그 24편 · YouTube 12편 · 블로그 · LinkedIn

[github.com/Youngdong2/claude-seminar-references](https://github.com/Youngdong2/claude-seminar-references)